Universidade Federal do Rio de Janeiro
Centro de Ciências Matemáticas e da Natureza
Instituto de Computação

P2 de Redes de Computadores 2025/1- Cada item de resposta precisa de *no mínimo* 2 frases de justificativa.

Assinatura: _____

Questão 1) (2,5 pontos) Considere o protocolo go-back-N. Assuma que o tamanho da janela do emissor é igual a J, e que os números de sequência variem entre 0 e S. Suponha que no instante de tempo t, o receptor esteja esperando um pacote com número de sequência k. Exceto no item b), assuma que o canal não pode re-ordenar pacotes. Cada item deve vir acompanhado de no mínimo 1 diagrama e no mínimo 2 frases.

a) Seja J=1. Quais os possíveis números de sequência dentro da janela do emissor, no instante de tempo t? Justifique claramente sua resposta.
b) Assuma que o canal pode re-ordenar pacotes. Seja J=1. Quais os possíveis números de sequência dentro da janela do emissor, no instante de tempo t? Justifique claramente sua resposta.
c) Seja J=2. Quais são os possíveis valores de ACK dos pacotes propagando do receptor para o emissor no instante de tempo t?
d) Explique um possível problema relacionado ao cenário S=1 e J=1.
e) Explique um possível problema relacionado ao cenário S=1 e J > 1.

Questão 2) (3,0 pontos) Sobre o TCP.

a) (1,0 ponto) O TCP se aproveita de características do go-back-N (GBN) e do Selective Repeat (SR). Explique uma das características que o TCP se aproveita de cada um desses protocolos, justificando. Qual a vantagem do GBN? Qual a vantagem do SR?
b) (0,5 ponto) Assuma que todos os pacotes enviados do emissor para o receptor têm tamanho MSS, e que o TCP encontra-se num estado tal que o tamanho da janela é igual a cwnd = J x MSS bytes. No próximo RTT, na fase de **additive-increase**, deseja-se aumentar o tamanho da janela do TCP em _____ maximum-segment-size (MSS) bytes. Em quantos bytes o TCP aumenta o tamanho da janela do emissor, a cada ACK recebido? Dê sua resposta em função de MSS e J. Explique claramente seu raciocínio.
c) (0,5 ponto) Repita o item acima para um RTT na fase **slow start.**
d) (1,0 ponto) No TCP: `Timeout_Interval = Estimated_RTT + 4*(Deviation_RTT)`

   Por que o número 4 na fórmula acima? Qual a desigualdade matemática que é usada para justificar esse valor? O `Timeout_Interval` não pode ser nem muito grande nem muito pequeno. O que ocorre se for muito grande? E se for muito pequeno?

Questão 3) (3 pontos) Verdadeiro ou falso? Justifique claramente. No caso de estar certo, comente. No caso de estar errado, corrija completamente. Lembre-se que todas as questões da prova precisam ser justificadas com no mínimo duas frases completas. É absolutamente fundamental que cada item seja justificado com no mínimo duas frases, comentando sobre o assunto.

a) O NAT faz uso de portas para resolver o problema da escassez de IPs.
b) SDN, ao unificar o plano de dados com o plano de controle, simplifica o gerenciamento da rede.
c) O DHCP faz uso do UDP.
d) Uma subrede é um conjunto de interfaces que conseguem se comunicar diretamente entre si.
e) O IPv6 aumentou o número de bits do endereço IP de 32 para 128 e manteve as demais funcionalidades do IPv4.
f) Ao encaminhar pacotes, os roteadores clássicos usam longest prefix matching enquanto que os roteadores SDN podem ou não usar longest prefix matching.

Questão 4) (1,5 pontos) Considere um servidor de previsão do tempo que transmite, a cada minuto, um pacote contendo a temperatura, continuamente, em uma única conexão TCP, sem concorrência na rede. Não há nenhuma outra conexão, nem de TCP, nem de UDP, na rede. Discuta, para cada um dos algoritmos de controle de congestionamento a seguir, se é possível que, a partir de um certo instante em que a rede esteja estável, a conexão deixe de sofrer perdas para sempre. Em cada caso, justifique claramente sua resposta. Cada item deve contar com no mínimo 2 frases, e deve explicar com alguns detalhes como funciona cada um dos algoritmos. a) (0,5 ponto) TCP Reno; b) (0,5 ponto) TCP Cubic; c) (0,5 ponto) TCP BBR.

Questão 1 – Go-Back-N (2,5 pontos)

a) J = 1, sem reordenação de pacotes:
Resposta: Com janela J = 1, o emissor pode ter no máximo um pacote não confirmado. Se o receptor espera o pacote com número de sequência k, isso significa que ele já recebeu corretamente os pacotes de 0 a k−1.
Logo, a janela do emissor contém apenas o número de sequência k-1 ou k.

Justificativa:

1. No Go-Back-N, a janela do emissor se move com base nos ACKs cumulativos.

2. Como J = 1, o emissor só pode enviar um novo pacote após receber o ACK do anterior. Assim, ele só pode ter enviado o pacote k, ainda não confirmado, ou k-1, também não confirmado (confirmação chegando).

---

b) Qualquer coisa pode acontecer se existir reordenamento. Afinal, um pacote pode ficar rodando na rede por um tempo enorme e eventualmente chegar no receptor, ou vice versa, um ACK ficar rodando na rede e eventualmente chegar no emissor.

Assuma, por exemplo, que para cada pacote enviado pelo emissor, ele também enviou uma retransmissão prematura. As retransmissões prematuras chegam de tal forma que o receptor fique esperando o pacote k, mas o emissor pode estar em qualquer estado. O receptor não consegue distinguir entre retransmissão prematura e pacote original, e o emissor pode des-sincronizar arbitrariamente com o receptor.

Se a reordenação é permitida e o canal tem atraso arbitrário, o receptor pode estar esperando o pacote com número de sequência k, mas o emissor pode estar em diferentes estados, inclusive retransmitindo pacotes antigos com o mesmo número de sequência.

Nesse caso, o número de sequência na janela do emissor pode ser qualquer um do conjunto {0,1,...,S} pois a ausência de ordem e o atraso arbitrário tornam possível que o emissor tenha reiniciado a janela sem que o receptor tenha recebido ou reconhecido os pacotes anteriores.

Justificativa:

1. Como J = 1, o emissor só envia um pacote por vez, mas pode reenviar o mesmo número várias vezes ao longo do tempo, se os ACKs forem perdidos ou muito atrasados.

2. A reordenação pode fazer com que o receptor ainda esteja esperando um pacote antigo, enquanto o emissor já avançou (de forma errada ou por timeout) para o envio de outros pacotes — desde que S seja suficientemente grande.

3. Se S for pequeno (ex: S = 1), há ambiguidade de sequência, como discutido no item (d), o que piora ainda mais o cenário.

4. Logo, qualquer número de sequência pode aparecer na janela do emissor, desde que respeite a janela de tamanho 1.

---

c) J = 2. Possíveis valores de ACK em propagação:
Resposta: O receptor envia ACKs cumulativos. Se o último pacote recebido em ordem foi o de número k−1, então o receptor envia ACK k-1. O emissor pode estar com a janela

- (k-2, k-1) – ACKs k-2 e k-1 em trânsito (poderia até ter um ACK k-3 em trânsito!)
- (k-1, k) – ACK k-1 em trânsito (poderia até ter um ACK k-2 em trânsito!)
- (k, k+1) – nenhum ACK em trânsito (poderia até ter um ACK k-1 em trânsito!)

Podem estar em trânsito os ACKs k-3, k-2, k-1, dependendo de quando foram emitidos.

Se o aluno considerar que o ACK vai com o número de sequência do pacote esperado, a resposta seria k-2, k-1, ou k.

Justificativa:

1. ACKs podem demorar a chegar ao emissor.

2. Se o receptor envia sempre o ACK do próximo esperado, podem coexistir ACKs k-1, k no caminho.

---

d) Não há problema funcional em Go-Back-N com S=1 J=1, pois o receptor GBN aceita apenas o pacote com número de sequência esperado, descartando os demais.

Assim, mesmo que o número de sequência se repita, o receptor sabe exatamente o que deve aceitar e o que descartar, evitando ambiguidade.

O problema aqui é eficiência! Esse protocolo é o stop-and-wait. Quem falou que não tem problema também ganhou os pontos.

---

e) Problema com S = 1 e J > 1:
Resposta: O emissor pode enviar vários pacotes com o mesmo número de sequência antes de receber ACKs.
Isso quebra a premissa do Go-Back-N, que exige números de sequência distintos dentro da janela.

Justificativa:

1. Sem espaço suficiente na numeração, o receptor não pode distinguir os pacotes.

2. É necessário que S≥J **me parece que isso não é suficiente!!!**

Practice Problem 6) (Computer Networking: A Top-Down Approach 6th Edition: Chapter 3 P22)
Consider the GBN protocol with a sender window size of 4 and a sequence number range of 1,024. Suppose that at time t, the next in-order packet that the receiver is expecting has a sequence number of k. Assume that the medium does not reorder messages. Answer the following questions:
a. What are the possible sets of sequence numbers inside the sender's window at time t? Justify your answer.
Here we have a window size of N.

- Suppose the receiver has received packet k-1, and has ACKed that and all other preceding packets. If all of these ACK's have been received by sender, then sender's window is [k, k+N-1].
- Suppose next that none of the ACKs have been received at the sender. In this second case, the sender's window contains k-1 and the N packets up to and including k-1. The sender's window is thus [k-N,k-1].

By these arguments, the senders window is of size N and begins somewhere in the range [k-N,k].
b. What are all possible values of the ACK field in all possible messages currently propagating back to the sender at time t? Justify your answer.
If the receiver is waiting for packet k, then it has received (and ACKed) packet k-1 and the N-1 packets before that. If none of those N ACKs have been yet received by the sender, then ACK messages with values of [k-N,k-1] may still be propagating back.Because the sender has sent packets [k-N, k-1], it must be the case that the sender has already received an ACK for k-N-1. Once the receiver has sent an ACK for k-N-1 it will never send an ACK that is less than k-N-1. Thus the range of inflight ACK values can range from k-N-1 to k-1.

Estamos analisando um protocolo de envio confiável (tipo Go-Back-N ou Selective Repeat).
O **remetente** envia pacotes numerados (0,1,2,...) e o **receptor** responde com confirmações (ACKs).

---

1. O que significa "o receptor está esperando o pacote *k*"?

- Quer dizer que ele já recebeu todos os pacotes **anteriores** a *k*.
- Então ele já recebeu o pacote *k-1* e também os **N−1 pacotes anteriores**.
- Portanto, o receptor **já enviou ACKs para os pacotes [k−N, ..., k−1]**.

---

2. Onde estão esses ACKs?

- Eles já foram gerados pelo receptor, mas podem ainda estar **viajando pela rede** de volta para o remetente.
- Isso significa que o remetente pode ainda não ter recebido todos esses ACKs.

- Então, na prática, ACKs com valores **[k−N, ..., k−1]** podem estar “em trânsito”.

---

3. O que o remetente já sabe?

- Como o remetente já enviou os pacotes [k−N, ..., k−1], ele obrigatoriamente **já recebeu o ACK do pacote k−N−1** (senão ele não teria avançado a janela).
- Isso é uma regra do protocolo: só se avança a janela quando um ACK chega.

---

4. O que o receptor nunca mais vai fazer?

- Uma vez que o receptor enviou ACK para *k−N−1*, ele **nunca mais enviará um ACK menor que esse valor**.
- Ou seja: os ACKs “velhos” ficam descartados.

---

5. Conclusão: qual é o intervalo possível de ACKs “em voo”?

- O **menor ACK possível em trânsito** é *k−N−1* (o último que com certeza chegou no remetente).
- O **maior ACK possível em trânsito** é *k−1* (o último pacote recebido).

Linha do tempo (esquema simplificado):

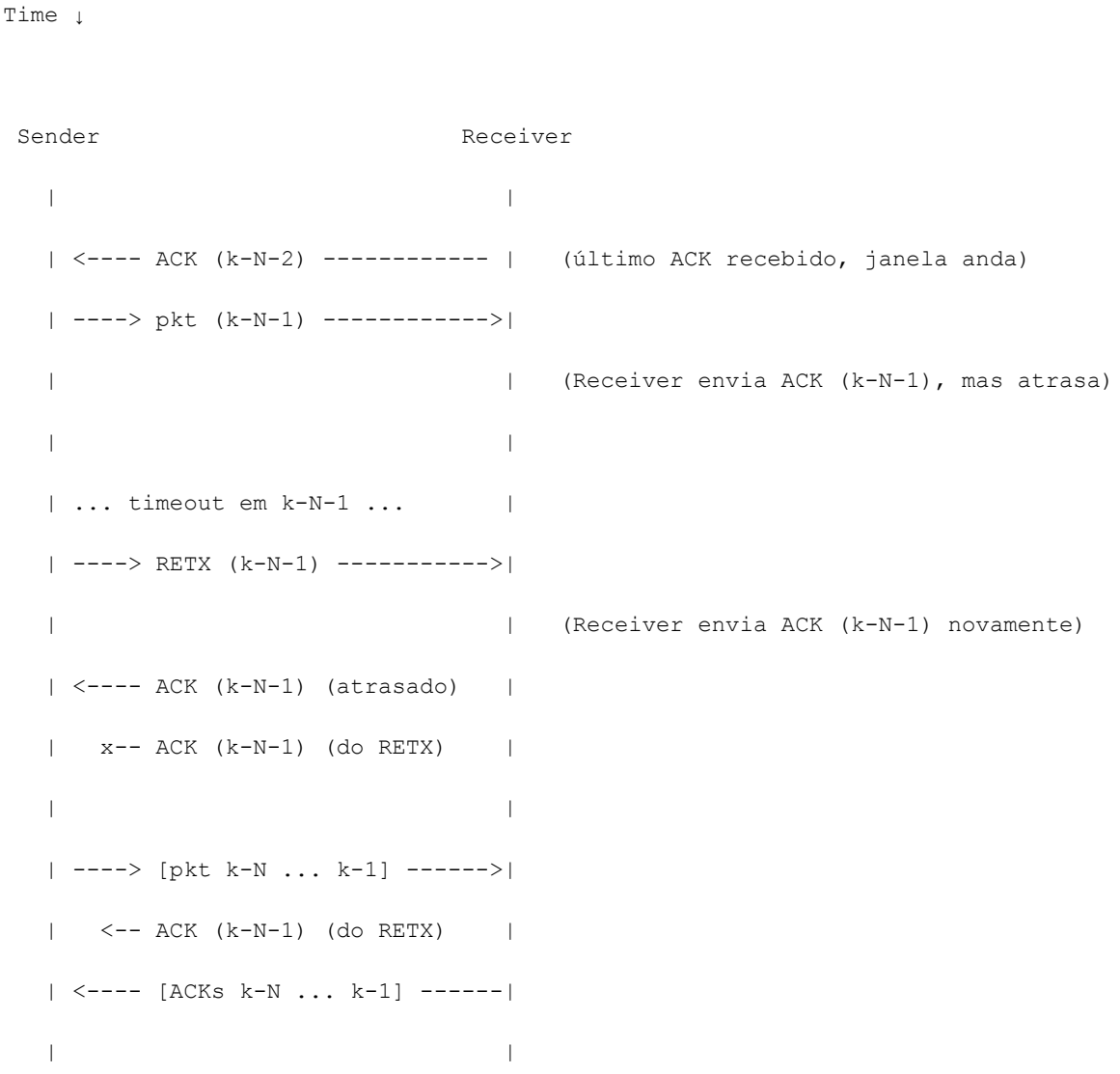
Caso 1)

Sender <---- ACK (k-N-1)

Sender ----> [pacotes k-N ... k-1] ----> Receiver

Sender <---- [ACKs k-N ... k-1] (podem ainda estar chegando)

Caso 2)



Por contradição, o ACK do pacote k-N-2 não poderia ainda estar trafegando. Caso estivesse, o ACK do pacote k-N-1 não teria sido recebido, e isso significa que o emissor não poderia ter enviado os pacotes de k-N até k-1

Interessante que essa questão acima também resolve essas abaixo:

P23. Consider the GBN and SR protocols. Suppose the sequence number space is of size k. What is the largest allowable sender window that will avoid the occurrence of problems such as that in Figure 3.27 for each of these protocols?

- **W<=quantidade de números de sequência/2 funciona**

    - **note que voltando para o exemplo J=2, S=1, ou seja, números de sequencia em {0,1}, nesse caso teríamos problema porque se emissor manda 2 pacotes, receptor recebe, passa janela pra frente pra {0,1} de novo, mas ACKs são perdidos, quando emissor retransmite o receptor não sabe se os pacotes são novos ou velhos**

- **2W<=quantidade de números de sequência**

Considere caso especial W=1, quantidade de números de seq. =2. Nesse caso, tudo OK.

P23. Consider the GBN and SR protocols. Suppose the sequence number space is of size k. What is the largest allowable sender window that will avoid the occurrence of problems such as that in Figure 3.27 for each of these protocols? Answer - The sender's window in Figure 3.27 is excessively big, which results in packets being lost or delayed, requiring retransmissions and decreasing efficiency. We need to establish the maximum permitted sender window for the SR and GBN protocols in order to prevent this issue. The maximum sender window for the GBN protocol is k/2. This is so that the sender can transmit up to k/2 unacknowledged packets without interfering with those that have already been sent. There is a chance that packets will overlap if the sender's window size is greater than k/2, which might confuse the receiver and lead to packet loss. In both protocols, the largest allowable sender window size is limited by the size of the sequence number space, as too large a window size can cause packet loss and decreased efficiency. By using a sender window size that is less than or equal to the largest allowable size, we can ensure that the protocols function correctly and efficiently

Answer: In order to avoid the scenario of Figure 3.27, we want to avoid having the leading edge of the receiver's window (i.e., the one with the “highest” sequence number)
wrap around in the sequence number space and overlap with the trailing edge (the one
with the "lowest" sequence number in the sender's window). That is, the sequence number space must be large enough to fit the entire receiver window and the entire sender window without this overlap condition. So we need to determine how large a range of sequence numbers can be covered at any given time by the receiver and sender
windows.
Suppose that the lowest-sequence number that the receiver is waiting for is packet m. In
this case, its window is [m, m+w-1] and it has received (and ACKed) packet m-1 and the

w-1 packets before that, where w is the size of the window. If the sender has yet received
none of those w ACKs, then ACK messages with values of [m-w, m-1] may still be propagating back. If the sender has received no ACKs with these ACK numbers, then the sender's window would be [m-w, m-1].
Thus, the lower edge of the sender's window is m-w, and the leading edge of the receiver's window is m+w-1. In order for the leading edge of the receiver's window to not overlap with the trailing edge of the sender's window, the sequence number space must thus be big enough to accommodate 2w sequence numbers. That is, the sequence
number space must be at least twice as large as the window size, k ≥ 2w.

P24. Answer true or false to the following questions and briefly justify your answer:
a. With the SR protocol, it is possible for the sender to receive an ACK for a packet that falls outside of its current window. ***TRUE***
b. With GBN, it is possible for the sender to receive an ACK for a packet that falls outside of its current window. ***TRUE***
c. The alternating-bit protocol is the same as the SR protocol with a sender and receiver window size of 1. ***TRUE***
d. The alternating-bit protocol is the same as the GBN protocol with a sender and receiver window size of 1. ***TRUE***

Questão 2)

O TCP se aproveita de características do Go-Back-N (GBN) e do Selective Repeat (SR). Explique uma característica que o TCP se aproveita de cada um desses protocolos, justificando. Qual a vantagem do GBN? Qual a vantagem do SR?

- **Do GBN**, o TCP herda a ideia de **envio de múltiplos pacotes sem esperar ACKs individuais**, ou seja, **uso de janela deslizante** para controle de fluxo.
    - **Vantagem do GBN**: é mais simples de implementar, pois o receptor só precisa manter o número do próximo pacote esperado, descartando qualquer pacote fora de ordem.
- **Do SR**, o TCP adota a estratégia de **retransmitir apenas os pacotes perdidos** (quando usado com mecanismos como SACK).
    - **Vantagem do SR**: **eficiência**, pois evita a retransmissão de pacotes que já foram corretamente recebidos, mesmo que estejam fora de ordem.

(0,5 ponto)

Atenção! Nessa questão a maior parte dos pontos foi para esse item – Em quantos bytes o TCP aumenta o tamanho da janela do emissor, a cada ACK recebido?

Assuma cwnd = J × MSS. Na fase de additive-increase, deseja-se aumentar a janela do TCP em _____ MSS por RTT. Em quantos bytes o TCP aumenta a janela a cada ACK recebido? Dê sua resposta em função de MSS e J.

- Na fase de **additive-increase**, o TCP aumenta a janela em **1 MSS por RTT**.
- Se a janela atual é de `cwnd = J × MSS`, então **aproximadamente J ACKs** são recebidos por RTT (um para cada segmento).
- Para aumentar **1 MSS por RTT**, o TCP aumenta a janela em:

  A cada ACK, o TCP aumenta a janela em MSS / J bytes = MSS (MSS / cwnd)
  - J é medido em segmentos,
  - cwnd em bytes

(0,5 ponto)

Atenção! Nessa questão a maior parte dos pontos foi para esse item – Em quantos bytes o TCP aumenta o tamanho da janela do emissor, a cada ACK recebido?

Repita o item acima para um RTT na fase slow start.

- Na fase de **slow start**, o TCP dobra o tamanho da janela a cada RTT. Isso significa um aumento de **1 MSS por ACK** (pois o total será J × MSS adicionais após J ACKs).

Resposta final:
A cada ACK, o TCP aumenta a janela em

MSS bytes

(1,0 ponto)

No TCP: Timeout_Interval = Estimated_RTT + 4 × Deviation_RTT

- **Por que o número 4? Qual a desigualdade matemática que justifica?**
- O número **4** vem da **Desigualdade de Chebyshev**, que diz que a probabilidade de um valor amostral estar mais do que k desvios da média é no máximo 1/k^2

Isso garante que o **Timeout_Interval** cubra a maior parte das variações de RTT, sem ser muito agressivo.

- **Se for muito grande**: o TCP demora demais a detectar perdas → **baixa eficiência e throughput**.
- **Se for muito pequeno**: o TCP pode interpretar variações naturais de RTT como perdas → **retransmissões desnecessárias**, **congestionamento artificial** e **redução de desempenho**.

Questão 3) (3 pontos) Verdadeiro ou falso? Justifique claramente. No caso de estar certo, comente. No caso de estar errado, corrija completamente. Lembre-se que todas as questões da prova precisam ser justificadas com no mínimo duas frases completas. É absolutamente fundamental que cada item seja justificado com no mínimo duas frases, comentando sobre o assunto.

a) O NAT faz uso de portas para resolver o problema da escassez de IPs
Verdadeiro

b) SDN, ao unificar o plano de dados com o plano de controle, simplifica o gerenciamento da rede.
Falso. Ele separa – fiquei surpreso por muitos terem errado essa!

c) O DHCP faz uso do UDP.
V, pra broadcast

d) Uma subrede é um conjunto de interfaces que conseguem se comunicar diretamente entre si.
V

e) O IPv6 aumentou o número de bits do endereço IP de 32 para 128 e manteve as demais funcionalidades do IPv4.
F – ele não manteve fragmentação

f) Ao encaminhar pacotes, os roteadores clássicos usam longest prefix matching enquanto que os roteadores SDN podem ou não usar longest prefix matching.
V

Questão 4) (1,5 pontos) Considere um servidor de previsão do tempo que transmite, a cada minuto, um pacote contendo a temperatura, continuamente, em uma única conexão TCP, sem concorrência na rede. Não há nenhuma outra conexão, nem de TCP, nem de UDP, na rede. Discuta, para cada um dos algoritmos de controle de congestionamento a seguir, se é possível que, a partir de um certo instante em que a rede esteja estável, a conexão deixe de sofrer perdas para sempre. Em cada caso, justifique claramente sua resposta. Cada item deve contar com no mínimo 2 frases, e deve explicar com alguns detalhes como funciona cada um dos algoritmos. a) (0,5 ponto) TCP Reno; b) (0,5 ponto) TCP Cubic; c) (0,5 ponto) TCP BBR.

a) TCP Reno
Não, mesmo em uma rede estável e sem concorrência, o TCP Reno pode continuar sofrendo perdas. Isso ocorre porque o Reno aumenta a janela de congestionamento linearmente (fase de *additive increase*) até ocorrer uma perda, o que faz com que ele interprete isso como sinal de congestionamento e reduza a janela pela metade (*multiplicative decrease*), iniciando um novo ciclo. Assim, o Reno tende a operar oscilando entre perda e recuperação, sem convergir para um ponto de equilíbrio estável.

b) TCP Cubic
Também não é garantido que o TCP Cubic pare de sofrer perdas permanentemente. O Cubic utiliza uma função cúbica do tempo desde a última perda para aumentar a janela, crescendo rapidamente após uma perda e desacelerando conforme se aproxima do valor anterior de janela. No entanto, esse comportamento leva o protocolo a continuamente "testar" a largura de banda da rede, o que pode causar novas perdas mesmo após longos períodos de estabilidade.

c) TCP BBR
Sim, é possível que o TCP BBR pare de sofrer perdas permanentemente, desde que as condições da rede permaneçam estáveis. O BBR não baseia o controle de congestionamento na ocorrência de perdas, mas sim em estimativas de largura de banda e *RTT* (round-trip time). Ao modelar a capacidade do caminho, o BBR tende a enviar apenas o volume de dados correspondente à banda estimada, o que pode eliminar as perdas após a fase de convergência do modelo.

Note que como a taxa de transmissão efetiva por parte da aplicação é muito pequena, pode ser que a perda nunca ocorra em nenhum caso. Isso porque mesmo que a janela de congestionamento cresça, ainda assim o protocolo estará no regime não saturado, em que apenas um pacote estará trafegando na rede de cada vez. O fato de ser janela deslizante fará pouca diferença aqui. Entretanto, se aumentarmos a taxa de transmissão de dados por parte da aplicação, o argumento envolvendo a relação entre tamanho da janela e taxa de tranmissão se aplica.